

1. Write a C program that contains a string (char pointer) with a value "Hello world". The program should XOR each character in this string with 0 and displays the result.

Source code:

```
#include <stdio.h>

int main()

{ char *str = "Hello world";

    int i; printf("Original string: %s\n",

str); printf("Result after XOR with

0: "); for (i = 0; str[i] != '\0'; i++)

{ char result = str[i] ^ 0;      // XOR

    with 0

    printf("%c", result);

}

printf("\n");

return 0;

}
```

OUTPUT:

Original string: Hello world

Result after XOR with 0: Hello world

2. Write a C program that contains a string (char pointer) with a value "Hello world". The program should AND or and XOR each character in this string with 127 and display the result.

Source code:

```
#include <stdio.h>

#include <string.h>

int main()
```

```
{ char *str = "Hello world"; int i, len; len
= strlen(str); printf("Result after AND
with 127:\n"); for (i = 0; i < len; i++)
{ char and_result = str[i] & 127;
printf("%c", and_result);
} printf("\n\nResult after XOR with
127:\n"); for (i = 0; i < len; i++)
{ char xor_result = str[i] ^ 127;
printf("%c", xor_result);
}
printf("\n");
return 0;
}
```

OUTPUT:

Result after AND with 127:

Hello world

Result after XOR with 127:

.. . .

3) Write a Java program to perform encryption and decryption using the following algorithms

a) Caesar cipher Source Code:

```
import java.util.Scanner; public class
CaesarCipher { public static String
encrypt(String text, int shift) { StringBuilder
result = new StringBuilder();
for (char c : text.toCharArray())
{ if (c >= 'A' && c <= 'Z') {
char ch = (char) ((c - 'A' + shift) % 26 + 'A');
    result.append(ch);
}
else if (c >= 'a' && c <= 'z') { char ch =
(char) ((c - 'a' + shift) % 26 + 'a');
    result.append(ch);
} else
{ result.append(c);
}
}
return result.toString();
}
public static String decrypt(String text, int shift)
{ return encrypt(text, 26 - shift); // reverse shifting
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter text: ");
    String text = sc.nextLine();
    System.out.print("Enter shift value: ");
    int shift = sc.nextInt();
    String encrypted = encrypt(text, shift);
    String decrypted = decrypt(encrypted, shift);
    System.out.println("\nEncrypted Text: " + encrypted);
    System.out.println("Decrypted Text: " + decrypted); }
}
```

Output:

Enter text: pavan

Enter shift value: 3

Encrypted Text: sdydq

Decrypted Text: pavan

3(b) Substitution cipher Source

Code:

```
import java.util.*; public class
SubstitutionCipher {
```

```
static String key = "QWERTYUIOPASDFGHJKLZXCVBNM";
static String reverseKey = getReverseKey(key); public static
String getReverseKey(String key) {
char[] reverse = new char[26];
for (int i = 0; i < 26; i++) { char
c = key.charAt(i); reverse[c -
'A'] = (char) ('A' + i);
} return new
String(reverse);
} public static String encrypt(String text)
{
    StringBuilder result = new StringBuilder();
    for (char c : text.toCharArray()) { if (c >= 'A'
&& c <= 'Z') {
result.append(key.charAt(c - 'A'));
    } else if (c >= 'a' && c <= 'z') { char upper =
Character.toUpperCase(c); char encrypted =
key.charAt(upper - 'A');
result.append(Character.toLowerCase(encrypted));
    } else {
        result.append(c);
    }
}
return result.toString();
}
public static String decrypt(String text)
{
    StringBuilder result = new StringBuilder();

    for (char c : text.toCharArray()) {
        if (c >= 'A' && c <= 'Z') {
            result.append(reverseKey.charAt(c - 'A'));
        }
        else if (c >= 'a' && c <= 'z') {
            char upper = Character.toUpperCase(c); char
            decrypted = reverseKey.charAt(upper - 'A');
            result.append(Character.toLowerCase(decrypted));
        }
        else
        {
            result.append(c);
        }
    }
    return result.toString();
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Substitution Cipher Example");
    System.out.print("Enter text: ");
```

```
String text = sc.nextLine();
String encrypted = encrypt(text);
String decrypted = decrypt(encrypted);
System.out.println("\nSubstitution Key : " + key);
System.out.println("Reverse Key    : " + reverseKey);
System.out.println("\nEncrypted Text: " + encrypted);
System.out.println("Decrypted Text: " + decrypted);
}
```

} Output:

Substitution Cipher Example

Enter text: pavan

Substitution Key : QWERTYUIOPASDFGHJKLZXCVBNM

Reverse Key : KXVMCNOHPQRSZYIJADLEGWBUFT

Encrypted Text: hqcqf

Decrypted Text: pavan

3.C. Hill Cipher

```
import java.util.*; public class
HillCipher { public static int
charToNum(char c) { return c - 'A'; }
    public static char numToChar(int n) {
        return (char) (n + 'A');
    } public static String encrypt(String plaintext, int[][] key)
{
    plaintext = plaintext.toUpperCase().replaceAll("[^A-Z]", "");
    if (plaintext.length() % 2 != 0) {
        plaintext += "X";
    }
    StringBuilder ciphertext = new StringBuilder();
    for (int i = 0; i < plaintext.length(); i += 2) { int
    p1 = charToNum(plaintext.charAt(i)); int p2 =
    charToNum(plaintext.charAt(i+1)); int c1 =
    (key[0][0] * p1 + key[0][1] * p2) % 26; int c2 =
    (key[1][0] * p1 + key[1][1] * p2) % 26;
    ciphertext.append(numToChar(c1));
    ciphertext.append(numToChar(c2));
    }
    return ciphertext.toString();
}
    public static int modInverse(int a) {
        for (int i = 0; i < 26; i++) {
            if ((a * i) % 26 == 1)
                return i; } return -1; }
    public static String decrypt(String ciphertext, int[][] key) {
        ciphertext = ciphertext.toUpperCase().replaceAll("[^A-Z]", "");
        int det = key[0][0] * key[1][1] - key[0][1] * key[1][0];
        det = (det % 26 + 26) % 26;
        int detInv = modInverse(det);
        if (detInv == -1) {
            return "Key matrix is not invertible!";
```

```

    }
    int[][] invKey = new int[2][2]; invKey[0][0] = ( key[1][1]
    * detInv) % 26; invKey[1][1] = ( key[0][0] * detInv) %
    26; invKey[0][1] = (-key[0][1] * detInv + 26) % 26;
    invKey[1][0] = (-key[1][0] * detInv + 26) % 26;
    StringBuilder plaintext = new StringBuilder(); for (int i =
    0; i < ciphertext.length(); i += 2) { int c1 =
    charToNum(ciphertext.charAt(i)); int c2 =
    charToNum(ciphertext.charAt(i+1)); int p1 =
    (invKey[0][0] * c1 + invKey[0][1] * c2) % 26; int p2 =
    (invKey[1][0] * c1 + invKey[1][1] * c2) % 26;
    plaintext.append(numToChar(p1));
    plaintext.append(numToChar(p2));
    }
    return plaintext.toString();
}

public static void main(String[] args)
{ Scanner sc = new Scanner(System.in);
  int[][] key = {
    {3, 3},
    {2, 5}
  };
  System.out.println("Hill Cipher (2×2)");
  System.out.print("Enter plaintext: ");
  String plaintext = sc.nextLine();
  String encrypted = encrypt(plaintext, key);
  String decrypted = decrypt(encrypted, key);
  System.out.println("\nKey Matrix:");
  System.out.println("[3 3]");
  System.out.println("[2 5]");
  System.out.println("\nEncrypted Text: " + encrypted);
  System.out.println("Decrypted Text: " + decrypted);
}
} Output:

```

Hill Cipher (2?2)

Enter plaintext:

pavan Key Matrix:

[3 3]

[2 5]

Encrypted Text: TELQEL

Decrypted Text: PAVANX

4. Write a C/JAVA program to implement the DES algorithm logic.

Program:

```
import java.util.*; public
class SimpleDES {
    // --- Permutation and S-box tables (standard DES) ---
    // Initial Permutation (IP)
    private static final int[] IP = {
        58,50,42,34,26,18,10,2,
        60,52,44,36,28,20,12,4,
        62,54,46,38,30,22,14,6,
        64,56,48,40,32,24,16,8,
        57,49,41,33,25,17,9,1,
        59,51,43,35,27,19,11,3,
        61,53,45,37,29,21,13,5,
        63,55,47,39,31,23,15,7
    };
    // Final Permutation (IP^-1)
    private static final int[] FP = {
        40,8,48,16,56,24,64,32,
        39,7,47,15,55,23,63,31,
        38,6,46,14,54,22,62,30,
        37,5,45,13,53,21,61,29,
        36,4,44,12,52,20,60,28,
        35,3,43,11,51,19,59,27,
        34,2,42,10,50,18,58,26,
        33,1,41,9,49,17,57,25
    };
    // PC-1 (64 -> 56) private static
    final int[] PC1 = {
        57,49,41,33,25,17,9,
        1,58,50,42,34,26,18,
        10,2,59,51,43,35,27,
        19,11,3,60,52,44,36,
        63,55,47,39,31,23,15,
```

```
7,62,54,46,38,30,22,
14,6,61,53,45,37,29,
21,13,5,28,20,12,4
};
// PC-2 (56 -> 48) private static
final int[] PC2 = {
    14,17,11,24,1,5,
    3,28,15,6,21,10,
    23,19,12,4,26,8,
    16,7,27,20,13,2,
    41,52,31,37,47,55,
    30,40,51,45,33,48,
    44,49,39,56,34,53,
    46,42,50,36,29,32
};
// Number of left shifts per round
private static final int[] SHIFTS = {
    1,1,2,2,2,2,2,2,
    1,2,2,2,2,2,2,1
};
// Expansion E (32 -> 48)
private static final int[] E = {
    32,1,2,3,4,5,
    4,5,6,7,8,9,
    8,9,10,11,12,13,
    12,13,14,15,16,17,
    16,17,18,19,20,21,
    20,21,22,23,24,25,
    24,25,26,27,28,29,
    28,29,30,31,32,1
};
// S-boxes S1..S8 private static final
int[][][] SBOX = {
    { // S1 {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
```

```
{0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
{4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
{15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
},
{ // S2
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},
{ // S3
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},
{ // S4
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},
{ // S5
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
{14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
{4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
{11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
},
{ // S6
{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
{10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
{9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
{4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
},
```

```

{ // S7
    {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
},
{ // S8
    {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
    {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
    {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
    {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
}
};

// P permutation (32 -> 32)
private static final int[] P = {
    16,7,20,21,
    29,12,28,17,
    1,15,23,26,
    5,18,31,10,
    2,8,24,14,
    32,27,3,9,
    19,13,30,6,
    22,11,4,25
};

// --- Utility: convert hex string to bit array and back --private
static int[] hexToBits64(String hex) {
    if (hex.length() < 16) {
        hex = String.format("%16s", hex).replace(' ', '0');
    }
    hex = hex.substring(hex.length()-16); // ensure 16 chars
    long val = Long.parseUnsignedLong(hex, 16); int[] bits
    = new int[64]; for (int i = 0; i < 64; i++) {
        bits[i] = (int)((val >> (63 - i)) & 1L);
    }
}

```

```
        return bits; }
private static String bits64ToHex(int[] bits) {
    long val = 0; for (int i = 0;
        i < 64; i++) {
        val = (val << 1) | (bits[i] & 1);
    }
    return String.format("%016X", val);
}
// Generic permutation helper: permute bits[] by table (1-based indexes)
private static int[] permute(int[] input, int[] table) {
    int[] out = new int[table.length]; for
    (int i = 0; i < table.length; i++) {
        out[i] = input[table[i] - 1];
    } return
    out;
}
// Left rotate an int[] representing bits (size n) by shift positions
private static int[] leftRotate(int[] in, int shift) {
    int n = in.length; int[] out
    = new int[n]; for (int i =
    0; i < n; i++) {
        out[i] = in[(i + shift) % n];
    } return
    out;
}
// Key schedule: produce 16 subkeys (each 48 bits)
private static int[][] generateSubkeys(int[] key64)
{ int[] key56 = permute(key64, PC1); // 56 bits int[]
C = Arrays.copyOfRange(key56, 0, 28); int[] D =
Arrays.copyOfRange(key56, 28, 56); int[][]
subkeys = new int[16][48];

    for (int round = 0; round < 16; round++) {
```

```

        C = leftRotate(C, SHIFTS[round]);
        D = leftRotate(D, SHIFTS[round]);
        int[] CD = new int[56];
        System.arraycopy(C, 0, CD, 0, 28);
        System.arraycopy(D, 0, CD, 28, 28); subkeys[round]
        = permute(CD, PC2); // 48-bit subkey
    } return
    subkeys;
}

// The DES Feistel (F) function: input: 32-bit R, 48-bit subkey -> 32-bit output
private static int[] feistel(int[] R, int[] subkey) {
    // Expansion E
    int[] expanded = permute(R, E); // 48 bits
    // XOR with subkey for
    for (int i = 0; i < 48; i++) {
        expanded[i] ^= subkey[i];
    }
    // S-box substitution: 8 blocks of 6 -> 4 bits
    int[] sOutput = new int[32]; for (int s = 0; s
    < 8; s++) {
        int base = s * 6;
        int row = (expanded[base] << 1) | expanded[base + 5];
        int col = (expanded[base + 1] << 3) | (expanded[base + 2] << 2) | (expanded[base + 3] << 1) |
        expanded[base + 4]; int val =
        SBOX[s][row][col]; int outBase = s *
        4; sOutput[outBase] = (val >> 3)
        & 1; sOutput[outBase + 1] = (val >> 2)
        & 1; sOutput[outBase + 2] = (val >> 1)
        & 1; sOutput[outBase + 3] = (val) & 1;
    }
    // P permutation
    int[] fOut = permute(sOutput, P); // 32 bits
    return fOut;
}

```

```
// Encrypt one 64-bit block (bits array length 64) with 16 subkeys
private static int[] encryptBlock(int[] block64, int[][] subkeys) {
    int[] ip = permute(block64, IP); int[] L =
    Arrays.copyOfRange(ip, 0, 32); int[] R =
    Arrays.copyOfRange(ip, 32, 64); for (int
    round = 0; round < 16; round++) {
        int[] f = feistel(R, subkeys[round]);
        int[] newR = new int[32]; for (int i
        = 0; i < 32; i++) {
            newR[i] = L[i] ^ f[i];
        }
        L = R;
        R = newR;
    }
    // Preoutput: R then L (note the swap) int[]
    preout = new int[64];
    System.arraycopy(R, 0, preout, 0, 32);
    System.arraycopy(L, 0, preout, 32, 32);
    int[] cipher = permute(preout, FP);
    return cipher;
}

// Decrypt one 64-bit block with subkeys (reverse order) private
static int[] decryptBlock(int[] block64, int[][] subkeys) {
    int[] ip = permute(block64, IP); int[] L =
    Arrays.copyOfRange(ip, 0, 32); int[] R =
    Arrays.copyOfRange(ip, 32, 64); for (int
    round = 15; round >= 0; round--) {
        int[] f = feistel(R, subkeys[round]);
        int[] newR = new int[32]; for (int i
        = 0; i < 32; i++) {
            newR[i] = L[i] ^ f[i];
        }
        L = R;
        R = newR;
    }
}
```

```
}
int[] preout = new int[64];
System.arraycopy(R, 0, preout, 0, 32);
System.arraycopy(L, 0, preout, 32, 32);
int[] plain = permute(preout, FP);
return plain;
}
// High-level helpers: encrypt/decrypt hex-string block with hex key public
static String encryptHexBlock(String hexPlain64, String hexKey64) {
    int[] block = hexToBits64(hexPlain64); int[]
    key = hexToBits64(hexKey64); int[][] subkeys
    = generateSubkeys(key); int[] cipherBits =
    encryptBlock(block, subkeys); return
    bits64ToHex(cipherBits);
}
public static String decryptHexBlock(String hexCipher64, String hexKey64) {
    int[] block = hexToBits64(hexCipher64); int[]
    key = hexToBits64(hexKey64); int[][] subkeys
    = generateSubkeys(key); int[] plainBits =
    decryptBlock(block, subkeys); return
    bits64ToHex(plainBits);
}
// --- Simple demo / test with a known vector --public
static void main(String[] args) {
    // Example test (from standard DES example):
    // Plaintext: 0123456789ABCDEF
    // Key: 133457799BBCDFF1 //
    Cipher: 85E813540F0AB405
    String plainHex = "0123456789ABCDEF";
    String keyHex    = "133457799BBCDFF1";
    System.out.println("Plaintext : " + plainHex);
    System.out.println("Key      : " + keyHex);
    String cipher = encryptHexBlock(plainHex, keyHex);
    System.out.println("Encrypted : " + cipher + " (expected 85E813540F0AB405)");
}
```

```

String decrypted = decryptHexBlock(cipher, keyHex);
System.out.println("Decrypted : " + decrypted + " (expected 0123456789ABCDEF)");
// You can also accept user input (uncomment below to allow interactive mode)
/*
Scanner sc = new Scanner(System.in);
System.out.print("Enter 64-bit plaintext hex (16 hex chars): ");
String userPlain = sc.nextLine().trim();
System.out.print("Enter 64-bit key hex (16 hex chars): ");
String userKey = sc.nextLine().trim();
String enc = encryptHexBlock(userPlain, userKey); System.out.println("Encrypted:
" + enc);
String dec = decryptHexBlock(enc, userKey);
System.out.println("Decrypted: " + dec); sc.close();
*/
}
}

```

Output:

Plaintext : 0123456789ABCDEF

Key : 133457799BBCDFF1

Encrypted : 85E813540F0AB405 (expected 85E813540F0AB405)

Decrypted : 0123456789ABCDEF (expected 0123456789ABCDEF)

5. Write a C/JAVA program to implement the Rijndael algorithm logic.

Program:

```

#include <stdio.h> unsigned
char state[4][4]; unsigned
char sbox[256] = {
0x63,0x7C,0x77,0x7B,0xF2,0x6B,0x6F,0xC5,
0x30,0x01,0x67,0x2B,0xFE,0xD7,0xAB,0x76
};
void AddRoundKey(unsigned char key[4][4])
{
int i, j; for(i=0;i<4;i++)
for(j=0;j<4;j++)
state[i][j] ^= key[i][j];
}

```

```
} void
SubBytes()
{
    int i, j; for(i=0;i<4;i++)
        for(j=0;j<4;j++)
            state[i][j] = sbox[state[i][j] % 16];
} void
ShiftRows()
{ unsigned char temp;
    temp = state[1][0];
    state[1][0]=state[1][1];
    state[1][1]=state[1][2];
    state[1][2]=state[1][3];
    state[1][3]=temp;
    temp = state[2][0];
    state[2][0]=state[2][2];
    state[2][2]=temp;
    temp = state[2][1];
    state[2][1]=state[2][3];
    state[2][3]=temp;
    temp = state[3][3];
    state[3][3]=state[3][2];
    state[3][2]=state[3][1];
    state[3][1]=state[3][0];
    state[3][0]=temp; }
void DisplayState()
{
    int i, j; for(i=0;i<4;i++)
        { for(j=0;j<4;j++) printf("%02X
            ", state[i][j]);
          printf("\n");
        } } int
main()
```

```
{ int i, j, k = 0; unsigned char
plaintext[16] = {
    0x32,0x43,0xF6,0xA8,
    0x88,0x5A,0x30,0x8D,
    0x31,0x31,0x98,0xA2,
    0xE0,0x37,0x07,0x34
};
unsigned char key[4][4] = {
    {0x2B,0x28,0xAB,0x09},
    {0x7E,0xAE,0xF7,0xCF},
    {0x15,0xD2,0x15,0x4F},
    {0x16,0xA6,0x88,0x3C}
}; for(i=0;i<4;i++)
for(j=0;j<4;j++)

    state[j][i] = plaintext[k++];
printf("Initial State:\n"); DisplayState();
AddRoundKey(key);
SubBytes();
ShiftRows();
printf("\nState After One Rijndael Round:\n");
DisplayState(); return 0;
}
```

Output:

Initial State:

32 88 31 E0

43 5A 31 37

F6 30 98 07

A8 8D A2 34

State After One Rijndael Round:

01 63 67 01

F2 6F 30 D7

D7 30 7B 77

30 AB 2B 67

8. Write a Java program to implement RSA algorithm.

```
import java.util.Scanner; import
java.math.BigInteger; public class
RSA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Step 1: Input two prime numbers
        System.out.print("Enter first prime number (p): ");
        BigInteger p = sc.nextBigInteger();
        System.out.print("Enter second prime number (q): ");
        BigInteger q = sc.nextBigInteger();
        // Step 2: Compute n and phi(n)
        BigInteger n = p.multiply(q);
        BigInteger phi = (p.subtract(BigInteger.ONE))
            .multiply(q.subtract(BigInteger.ONE));
        // Step 3: Choose public key e
        BigInteger e = BigInteger.valueOf(2); while
        (e.compareTo(phi) < 0) {
            if (e.gcd(phi).equals(BigInteger.ONE))
                break;
            e = e.add(BigInteger.ONE);
        }
        // Step 4: Compute private key d
        BigInteger d = e.modInverse(phi);
        System.out.println("\nPublic Key (e, n): (" + e + ", " + n + ")");
        System.out.println("Private Key (d, n): (" + d + ", " + n + ")");
        // Step 5: Input message
        System.out.print("\nEnter message (number form): ");
        BigInteger msg = sc.nextBigInteger();
        // Step 6: Encryption:  $C = M^e \bmod n$ 
        BigInteger cipher = msg.modPow(e, n);
        System.out.println("Encrypted Message: " + cipher);
        // Step 7: Decryption:  $M = C^d \bmod n$ 
        BigInteger decrypted = cipher.modPow(d, n);
        System.out.println("Decrypted Message: " + decrypted); sc.close();
    }
}
```

Output:

```
Enter first prime number (p): 11
Enter second prime number (q): 13
```

```
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)
```

Enter message (number form): 9

Encrypted Message: 48

Decrypted Message: 9

9. Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript.

```
<!DOCTYPE html>
<html>
<head>
  <title>Diffie-Hellman Key Exchange</title>
  <style>
    body {
      font-family: Arial; background-
        color: #f2f2f2; padding: 20px;
    } h2
    {
      text-align: center; color:
        darkblue;
    }
    .box { background: white; padding:
      20px; border-radius: 10px;
      width: 500px; margin: auto;
      box-shadow: 0px 0px 10px gray;
    } input
    {
      width: 100%;
      padding: 6px;
      margin: 6px 0;
    }
    button
    {
      width: 100%; padding:
        8px; background:
        darkblue; color: white;
        border: none; font-
        size: 16px; cursor:
        pointer;
    }
    .result { background:
      #e8f0fe; padding:
        10px; margin-top:
        10px; border-radius:
        6px;
    }
  </style>
</head>
<body>
  <h2>Diffie-Hellman Key Exchange Demo</h2>
  <div class="box">
    <label>Prime Number (p):</label>
    <input type="number" id="p" value="23">
```

```

<label>Primitive Root (g):</label>
<input type="number" id="g" value="5">
<label>Alice's Private Key (a):</label>
<input type="number" id="a" value="6">
<label>Bob's Private Key (b):</label>
<input type="number" id="b" value="15">
<button onclick="calculate()">Generate Keys</button>
<div class="result" id="output"></div>
</div> <script> function
powerMod(base, exp, mod) {
  let result = 1; for (let i = 1; i
  <= exp; i++) {
    result = (result * base) % mod;
  }
  return result; }
function calculate()
{
  let p = parseInt(document.getElementById("p").value);
  let g = parseInt(document.getElementById("g").value);
  let a = parseInt(document.getElementById("a").value);
  let b = parseInt(document.getElementById("b").value);
  // Public keys let A = powerMod(g, a, p); let B =
  powerMod(g, b, p); // Shared secret keys let
  keyAlice = powerMod(B, a, p); let keyBob =
  powerMod(A, b, p);
  document.getElementById("output").innerHTML
  =
    "<b>Alice sends:</b> A = " + A + "<br>" +
    "<b>Bob sends:</b> B = " + B + "<br><br>" +
    "<b>Alice's Secret Key:</b> " + keyAlice + "<br>" +
    "<b>Bob's Secret Key:</b> " + keyBob + "<br><br>" +
    "<b>Shared Secret Key:</b> " + keyAlice;
}
</script>
</body>
</html>

```

Output:

Diffie-Hellman Key Exchange Demo

Prime Number (p):

Primitive Root (g):

Alice's Private Key (a):

Bob's Private Key (b):

Generate Keys

Alice sends: $A = 8$

Bob sends: $B = 19$

Alice's Secret Key: 2

Bob's Secret Key: 2

Shared Secret Key: 2